

THE UNIVERSITY OF
SYDNEY

The University of Sydney

Aerospace, Mechanical and Mechatronic Engineering

MTRX1705 Project 1

SID: 520481220

An Assignment submitted for the UoS:

Introduction to Mechatronic Design

September 21, 2025

Contents

1	Design Process	1
1.1	State Machine	1
1.1.1	State Transition Diagram	1
1.1.2	Truth Table	2
1.1.3	Clock	2
1.1.4	Memory	2
1.2	Output Logic	2
1.2.1	Truth Table	2
1.2.2	Karnaugh Maps	3
1.2.3	NAND and Inverter Form	3
1.3	Coin Mode	5
1.3.1	State Transition Diagram	5
1.3.2	Next-State Truth Table	5
1.3.3	Desired BCD Outputs	5
1.3.4	Output Logic Adjustment	5
1.4	Debug Mode	6
2	Final Circuit	7

1 Design Process

The aim of this project was to design and implement a digital circuit capable of simulating either a six-sided die or a coin toss, with outputs displayed on a seven-segment display. Randomness in the system is derived from the user's release timing of a pushbutton, ensuring unpredictable outcomes. The design needed to use only the parts provided in lab, minimise component count, and support debugging through slowed clock operation. Randomness is assumed to arise from human error in button release timing, and the system is considered closed, meaning no external influence affects state transitions. To preserve fairness, the user should not be aware of the current or starting state, and display outputs must correspond to die values 1–6 or coin values 1–2. To achieve this, a top-level block diagram [Figure 1](#) was created at the beginning of the process, dividing the circuit into three main modules: a clock module to generate timing signals for state transitions, a state machine module based on a 3-bit Johnson counter forming the core of the dice/coin sequence, and an output logic module to decode the current state into BCD signals for the seven-segment display. This modular design approach enabled independent development and debugging of each block before final integration.

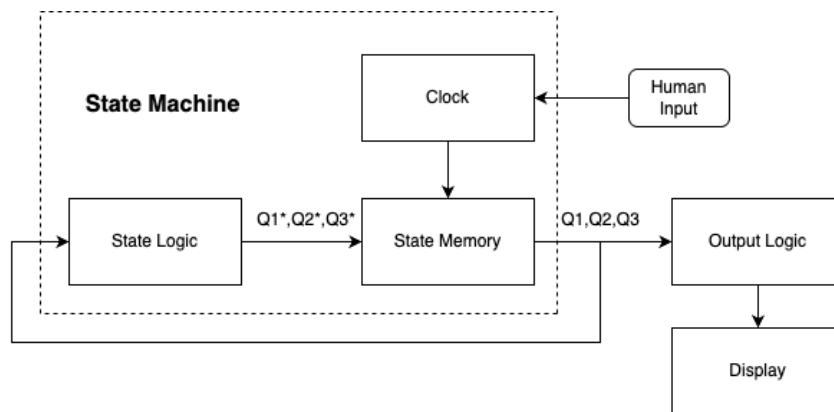


Figure 1: Top Down Design

1.1 State Machine

1.1.1 State Transition Diagram

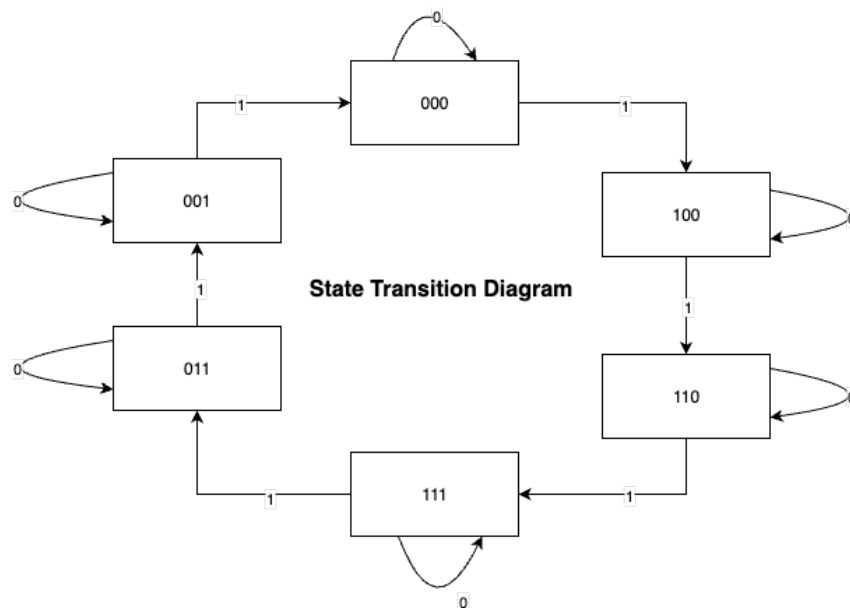


Figure 2: State transition diagram

The state transition diagram in [Figure 2](#) represents the operation of the 3-bit Johnson counter used as the state machine. The sequence cycles through six unique states corresponding which will correspond to the display outputs 1 through 6. A transition to the next state occurs on a clock signal of 1, while

a clock signal of 0 causes the machine to remain in its current state. This design ensures predictable progression through the required states while maintaining state when the clock is low.

1.1.2 Truth Table

Table 1: Truth table for next state transitions

Q1	Q2	Q3	Q1*	Q2*	Q3*
0	0	0	1	0	0
1	0	0	1	1	0
1	1	0	1	1	1
1	1	1	0	1	1
0	1	1	0	0	1
0	0	1	0	0	0

The next state behavior shown in [Table 1](#) follows directly from the properties of a 3-bit Johnson counter, which naturally cycles through six unique states. From this table, the simplified next state equations in [Equation 1](#) are obtained, where each flip flop is driven by either the output of another flip flop or the complement of Q_3 .

$$Q_1^* = \overline{Q_3}, \quad Q_2^* = Q_1, \quad Q_3^* = Q_2 \quad (1)$$

1.1.3 Clock

An essential sub module of the state machine is the clock, which in combination with flip-flops provides the memory required for sequential operation. The clock was implemented using a 555 timer circuit, configured in astable mode with resistors of $470\ \Omega$ and $470\ \Omega$, and a $100\ \mu\text{F}$ capacitor. This configuration produces a square wave at a frequency given by, and by substituting $R_A = 470\ \Omega$, $R_B = 470\ \Omega$, and $C = 100\ \mu\text{F}$,

$$f = \frac{1.44}{(R_A + 2R_B)C} = \frac{1.44}{(470 + 2 \cdot 470)(100 \times 10^{-6})} \approx 10.2\ \text{Hz}$$

A pushbutton was added to toggle power to the timer, allowing the clock to be enabled or disabled as required. The circuit design was validated and tested on the breadboard, confirming proper operation. For debugging purposes, the capacitor was replaced with a larger $220\ \mu\text{F}$, reducing the frequency to

$$f = \frac{1.44}{(470 + 2 \cdot 470)(220 \times 10^{-6})} \approx 4.6\ \text{Hz},$$

Which allowed each state transition to be observed more clearly. This dual capacitor approach provided both a practical operating frequency and a slower debug mode.

1.1.4 Memory

The final sub module of the state machine consists of a series of flip-flops that store the current state of the system until the next clock transition. Each flip-flop receives the corresponding next-state input (Q_1^*, Q_2^*, Q_3^*) and updates its output (Q_1, Q_2, Q_3) on the rising edge of the clock signal. These outputs are then fed back into the system as inputs for the subsequent state transition, ensuring the Johnson counter progresses through its sequence correctly.

1.2 Output Logic

1.2.1 Truth Table

The output logic converts the Johnson counter states (Q_1, Q_2, Q_3) into BCD inputs (A, B, C) for the seven-segment display. The fourth BCD input, D , is tied to logic low and is not included in the truth table. The mapping is shown in [Table 2](#).

Table 2: Truth table for output logic (BCD inputs to seven segment display)

Q_1	Q_2	Q_3	A	B	C	Display
0	0	0	1	0	0	1
1	0	0	0	1	0	2
1	1	0	1	1	0	3
1	1	1	0	0	1	4
0	1	1	1	0	1	5
0	0	1	0	1	1	6

1.2.2 Karnaugh Maps

The Karnaugh maps for outputs A , B , and C were created using Q_1 as the row variable and Q_2Q_3 as the column variable (Gray code order: 00, 01, 11, 10). Don't care states are included to allow minimal grouping.

	00	01	11	10
0	1	0	1	X
1	0	X	0	1

Simplified Boolean expression for A :

$$A = Q_2\overline{Q_3} + \overline{Q_1}(Q_2 + \overline{Q_3})$$

	00	01	11	10
0	0	1	0	X
1	1	X	0	1

Simplified Boolean expression for B :

$$B = \overline{Q_2}Q_3 + Q_1\overline{Q_3}$$

	00	01	11	10
0	0	1	1	X
1	0	X	1	0

Simplified Boolean expression for C :

$$C = Q_3$$

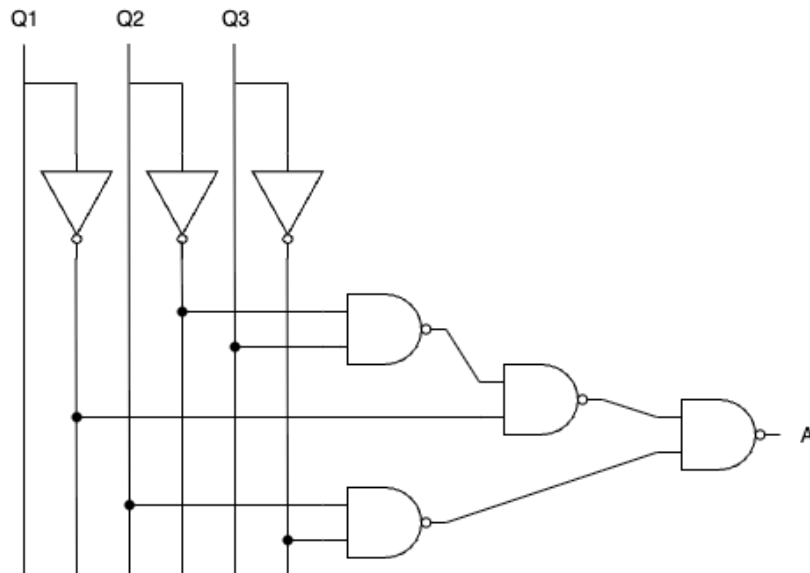
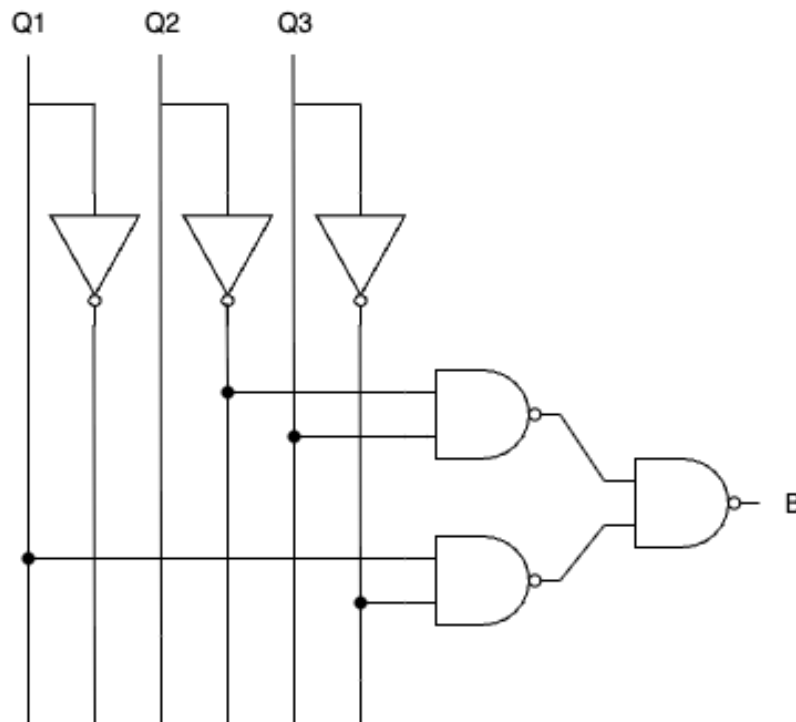
1.2.3 NAND and Inverter Form

The Boolean expressions for the outputs were converted into NAND and inverter form by systematically applying DeMorgan's laws. This approach allows the design to be implemented using only NAND gates and inverters, as provided in the lab, while preserving the original logic.

$$A = \left((\overline{Q_2}Q_3)' \cdot (\overline{Q_1} \cdot (Q_2\overline{Q_3})') \right)' \quad (2)$$

$$B = \left(\overline{\overline{Q_2}Q_3} \cdot \overline{Q_1\overline{Q_3}} \right)' \quad (3)$$

The output C is trivial and remains Q_3 , requiring no further conversion. These equations can be directly implemented in the output logic circuitry.

Figure 3: Circuit implementation of output A using NAND gates and inverters.Figure 4: Circuit implementation of output B using NAND gates and inverters.

The circuits for outputs A and B implement the simplified Boolean expressions using only NAND gates and inverters, in accordance with the lab components. Each NAND gate performs either a direct logical NAND operation or serves as an inverter when both inputs are the same signal. The wiring mirrors the hierarchical structure of the Boolean expressions: intermediate signals from the first level of NAND gates feed into subsequent gates, culminating in the final output. Output C is trivial and is connected directly to Q_3 without additional logic. These circuit implementations ensure that the Johnson counter states are correctly translated into the appropriate BCD signals for the seven-segment display, preserving the intended dice and coin display behavior.

1.3 Coin Mode

1.3.1 State Transition Diagram

Coin mode was implemented by modifying the Johnson counter to cycle through only two states instead of six. This was achieved using a CTS 208-4 switch, which when switch0 activated, feeds the complemented value of $\overline{Q_1}$ into the D1 input of the first flip-flop in the counter. As a result, the counter is constrained to a two state sequence, corresponding to the two sides of a coin (1 and 2).

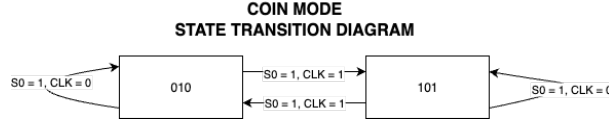


Figure 5: State transition diagram for coin mode.

1.3.2 Next-State Truth Table

Table 3: Next-state transitions for coin mode

S0	Q1	Q2	Q3	Q1*	Q2*	Q3*
1	0	1	0	1	0	1
1	1	0	1	0	1	0

$$Q1^* = \overline{Q2}, \quad Q2^* = Q3, \quad Q3^* = \overline{Q2} \quad (4)$$

In this two-state configuration, the previously defined output logic for A and B remains valid. However, $C = Q_3$ no longer produces the correct display for a coin, requiring a modification.

1.3.3 Desired BCD Outputs

Table 4: Next-state transitions with desired BCD outputs for coin mode

S0	Q1	Q2	Q3	A	B	C	Display
1	0	1	0	1	0	0	1
1	1	0	1	0	1	1	6
Desired BCD outputs are shown below							
1	0	1	0	1	0	0	1
1	1	0	1	0	1	0	2

1.3.4 Output Logic Adjustment

To achieve the desired BCD outputs, the logic for C was modified. Using Switch1, the C output is generated according to the following table:

Table 5: Truth Table for C

C	Switch 1	C
1	0	1
0	0	0
1	1	0
0	1	0

From this table, the logic for C can be expressed as:

$$C = Q_3 \cdot \overline{\text{Switch1}}$$

Implemented with NAND gates, this becomes:

$$C = \overline{\text{NAND}(Q_3, \overline{\text{Switch1}})}$$

Which gives the output logic circuit seen in [Figure 6](#).

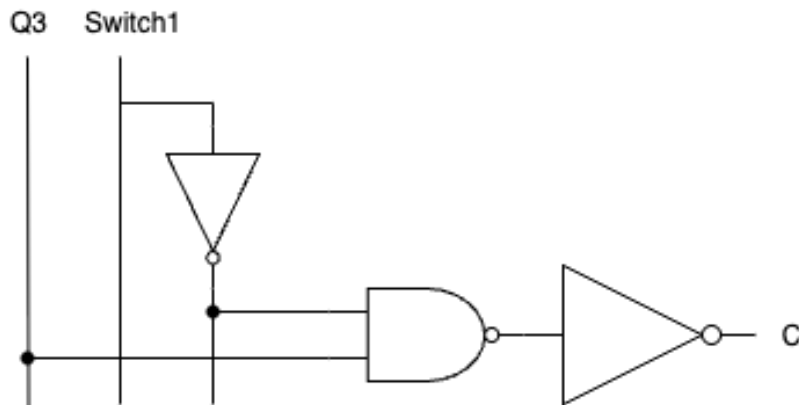


Figure 6: Circuit implementation of output C using NAND gates and inverters.

This modification is necessary because cutting the Johnson counter with Switch0 produces a reduced two-state transition ($Q_1Q_2Q_3 : 010 \rightarrow 001$). While A and B remain correct, the adjusted logic for C ensures the seven-segment display accurately represents the coin toss result. The pushbutton functionality from dice mode is preserved, so the outcome remains effectively random based on human reaction timing.

1.4 Debug Mode

To facilitate troubleshooting, the circuit was designed so that the clock speed could be adjusted by switching between $100\ \mu\text{F}$ and $220\ \mu\text{F}$ capacitors. This allowed the state changes to occur slowly enough to be easily observed. Each module, particularly the clock, was tested independently on the breadboard before integrating the full system, which helped identify and correct errors efficiently. A multimeter was also used at various points in the circuit to confirm that the signals matched the expected HIGH and LOW logic levels throughout the operation.

2 Final Circuit

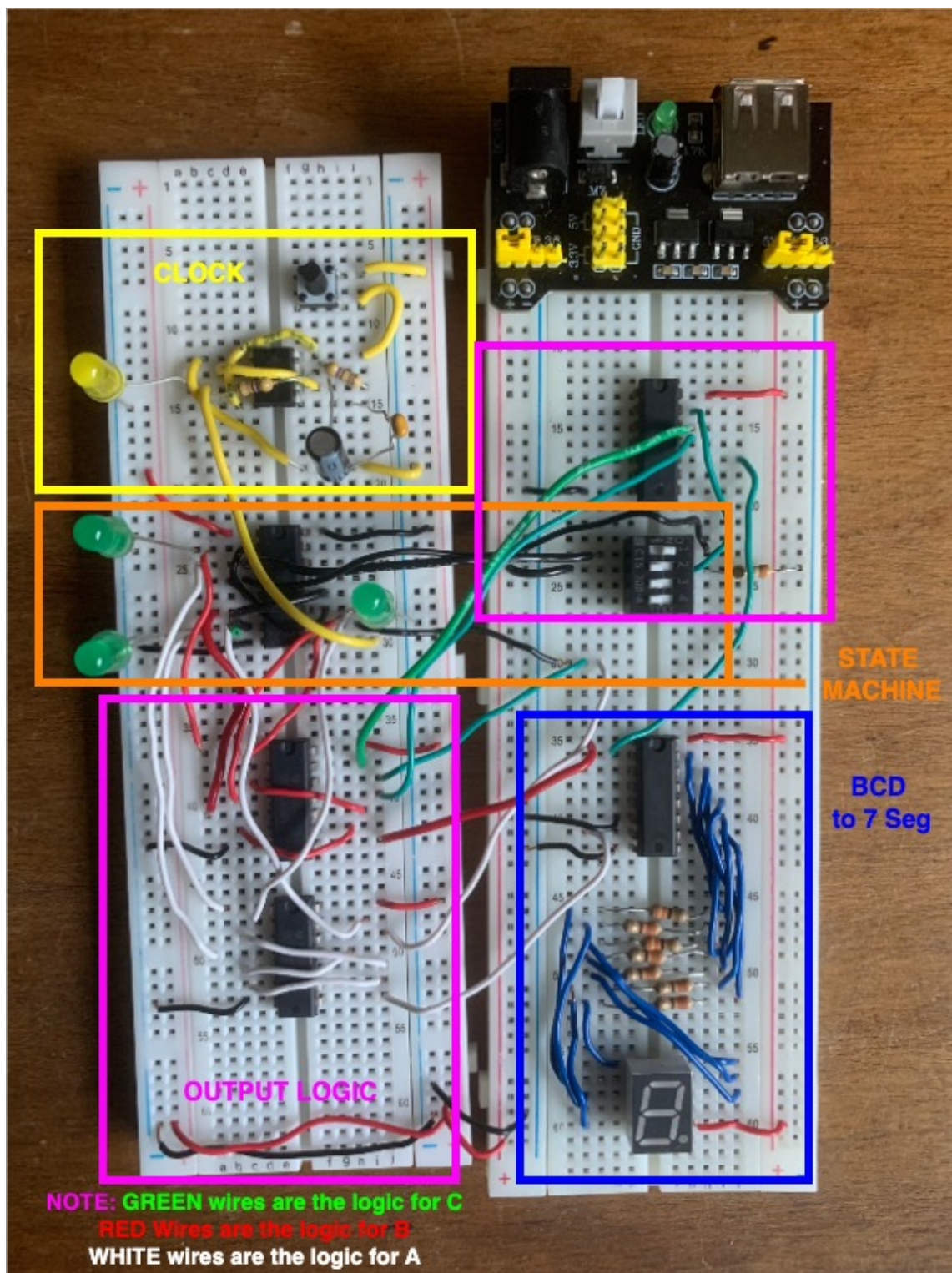


Figure 7: Final working circuit